# Tokenization vs Encoding vs Embedding

| Aspect | Tokenization | Encoding | Embedding |
|---|---|---|---|
| **Definition** | Breaking raw text into smaller units (words, subwords, or characters). | Converting tokens into numeric IDs + masks + special tokens so models can process them. | Mapping IDs into dense vectors of floats that carry semantic meaning. |
| **Input** | Raw text (e.g., "Unbelievable!") | Tokens (e.g., ["un", "believ", "able"]) | Input IDs (e.g., [101, 1045, 2293, ...]) |
| **Output** | Tokens (strings) | Numeric arrays:<br>– **input_ids** (IDs)<br>– **attention_mask** (padding info)<br>– **token_type_ids** (optional) | Dense vectors (continuous, learned). Shape: [batch_size, seq_len, hidden_dim]. |
| **Representation type** | Text units | Integer indices | Floating-point vectors |
| **Where meaning comes from** | No meaning, just splitting | No meaning, just indexing | Meaning arises here: embeddings capture similarity & semantics |
| **Granularity** | Word / Subword / Character | Sequence-level preparation (ids, masks, padding) | Vector-level semantic space |
| **Special handling** | Handles unknown words via subword splits | Adds [CLS], [SEP], [PAD]; ensures uniform length | Adds positional embeddings, combines with token embeddings |
| **Problem solved** | Text → machine-understandable chunks | Make tokens numeric & model-ready | Give numeric ids continuous, meaningful space to learn patterns |
| **Example** | "unbelievable" → ["un", "believe", "able"] | ["un","believe","able"] → [17953, 4567, 2468] (+ masks) | [17953,...] → [[0.12, -0.3, ...], [0.5, 0.2, ...], [0.09, -0.7, ...]] |
| **Common confusion** | Token ≠ numeric ID | IDs are *indexes*, not meanings | Input embedding vs contextual embedding (model output) |
| **Dependency** | Needs vocabulary rules (BPE, WordPiece, etc.) | Needs vocab mapping (token → id) | Needs embedding matrix (trainable weights) |